AI-Driven Adaptive Transaction Batching for Cardano Decentralized Exchanges

Vikranth Sai
Dept. of CSE (Data Science)
Vignana Bharathi Institute of Technology, Hyderabad, India
Email: vikranthsai310@gmail.com

Mrs. B. Mamatha
Dept. of CSE (Data Science)
Vignana Bharathi Institute of Technology, Hyderabad, India
Email: bale.mamatha@vbithyd.ac.in

Sowmya
Dept. of CSE (Data Science)
Vignana Bharathi Institute of Technology, Hyderabad, India
Email: sowmyavadla0@gmail.com

Ganesh
Dept. of CSE (Data Science)
Vignana Bharathi Institute of Technology, Hyderabad, India
Email: ganeshpothalanka@gmail.com

***Abstract*—A Cardano decentralized exchange keeps each liquidity pool in one UTXO, and only one transaction per block can spend it, so off-chain batchers settle many swaps together. Batchers in use today fire every M orders or T seconds without reading the chain, and because Cardano fees are fixed, a badly timed batch costs users time, not money. We treat batching as control of a shared pool lock under uncertain confirmation time. Over 388,781 mainnet blocks (92 days) the median block is only 3.0% full and just 0.56% pass 80%, so the bottleneck is the pool lock, not congestion. A discrete-event simulator replays these blocks to compare tuned static batchers with a LightGBM fill forecaster, a forecast-driven decision rule (P2) and a masked DQN agent behind a runtime safety shield. At matched load on held-out data, P2 cuts the fee per user by 18.9% against greedy submission for 5.1 s more average wait, and it is both faster and cheaper than the tuned interval batcher. No adaptive policy beats greedy on tail latency on today's lightly loaded chain. Without its shield the agent let 35.7% of orders expire under heavy load; with it, none expired. Chaining two batches nearly halves the heavy-load 95th-percentile wait, and a minimal Aiken exchange settled a batch from our batcher on the Cardano preprod testnet.**

***Index Terms*—Cardano, congestion forecasting, decentralized exchange, eUTXO, reinforcement learning, transaction batching**

I. INTRODUCTION

Cardano records ownership with the extended UTXO (eUTXO) model [1]. A decentralized exchange (DEX) keeps each liquidity pool in one UTXO that holds the pool's state, and every swap must spend it and create a new one. A UTXO can be spent only once, so at most one transaction per block can use a given pool: if a hundred users swap at once, one succeeds and the rest fail. Exchanges therefore rely on batchers, off-chain agents that collect the orders users have locked at an order address and settle many of them in one transaction that spends the pool once [2], [3].

The batchers in production fire on constants: every M orders, or every T seconds. They do not look at how full recent blocks were, how long the oldest order has waited, or how long their own batch is likely to take to confirm. This matters because a sent batch locks the pool until it is included. A badly timed batch delays every order behind it, a batch that is too small spreads the fixed fee over few users, and a batch that waits too long delays the users it holds.

Cardano's fees follow a fixed formula, with no bidding:

$$fee = 44s + 155{,}381 + 0.0577m + 0.0000721c \quad (1)$$

$$C_{\text{user}} = fee(n) / n \quad (2)$$

where s is the size in bytes, m and c are the memory and CPU-step units, n is the number of orders in the batch and fees are in lovelace. Equation (1) is the protocol's linear fee: a fixed base of 155,381 lovelace plus a charge for every byte and execution unit. Equation (2) divides that fee among the n orders of a batch, so the fixed part each user pays shrinks as the batch grows. A transaction costs the same on an idle chain and on a full one, so bad timing costs time, not money. The goal is therefore short waits and a low cost per user C_{user}, not low fees, which is why fee-market work for account-based chains such as Ethereum [4] does not carry over. The project began from a claim that Cardano blocks run 80–90% full. We tested it on 92 days of mainnet data before building on it, and it did not hold: the median block is 3.0% full (Section IV-A).

The problem is therefore narrower than a general call for intelligent batching: from the state of the queue and of the chain, decide at every block whether to send a batch and how many orders to put in it, when the shared resource is the pool lock, confirmation time is uncertain and protocol limits must never be broken. We make four contributions. First, a 92-day measurement showing that the bottleneck is the pool lock, not congestion. Second, a formulation of batching with two hard capacity gates. Third, a LightGBM forecaster, a forecast-driven decision rule and a masked DQN agent made safe by a runtime shield, together with batch chaining and one-price clearing. Fourth, an evaluation against tuned static batchers on held-out data, which finds that cost and fairness improve while tail latency does not beat greedy submission on today's chain. Section III describes the system and Sections IV and V evaluate it.

II. RELATED WORK

Chakravarty et al. define the eUTXO model, with datums, redeemers and validator scripts [1]. It gives fixed fees and local checks, but it also means that any state many users share sits in one UTXO and becomes a bottleneck. Nguyen Quang proposes batching in which each order is posted as its own UTXO and many orders are merged into one pool update [2]. This removes the contention, but the rule for when to batch is fixed. Bruhwiler

studies the same problem for AMM exchanges and builds a concurrent DEX prototype [3]; that work is about correctness, not about when to batch or how many orders to include.

Outside Cardano, Roughgarden studies the EIP-1559 fee mechanism, where a base fee rises with demand [4]. That model fits Ethereum but not Cardano, where a busy chain causes delay, not a higher price. Mao et al. show that deep reinforcement learning can learn job-scheduling policies as good as hand-tuned rules [5], but for datacenters, with no block limits and no pool lock. Angeris and Chitra study constant-function market makers, the pool model our batcher trades against [6]. Table I sums up the comparison.

TABLE I. COMPARISON OF THE RELATED WORK.

Work	Approach	Limitation here
eUTXO model [1]	Formal ledger specification	Single-UTXO concurrency bottleneck
MELD batching [2]	Deterministic order batching	Static policy; not congestion-aware
Concurrent DEX [3]	Concurrent DEX prototype	No batch timing or sizing
EIP-1559 [4]	Fee-mechanism analysis	Inapplicable to fixed fees
DeepRM [5]	Deep RL scheduling	Not blockchain-aware

Table I shows that earlier work either fixes the batching rule, ignores the state of the chain, or optimizes fees, which is the wrong goal on Cardano. No published method sets batch size or timing from the forecast capacity and the queue, and we found no batcher that learns from chain conditions to cut waiting time and per-user cost within Cardano's limits.

III. SYSTEM DESIGN

A. Problem Formulation

At each block slot the batcher observes the queue of pending orders (depth, oldest wait, and the size and execution units of each order), whether the pool is locked, and a forecast of how full the next blocks will be. It chooses WAIT or SUBMIT(n), the first n orders of the queue. A submitted batch locks the pool until it is included or its validity interval expires; while it is in flight there is nothing to decide and orders pile up behind it. The goal is to minimize waiting time and C_{user} under three constraints: the per-transaction limits (Gate A), the predicted room in the next block (Gate B), and a deadline $D_{\max}$ on the oldest order.

B. Architecture

The system is an event-driven control loop around a forecast-then-decide pipeline, with an offline simulator that can stand in for the live chain (Fig. 1). The batcher is a server-side process, because a wallet sees only its owner's funds and aggregation is the batcher's job. A collector (M1) reads blocks into a feature store, a forecaster (M2) predicts block fill, a queue manager (M3) tracks pending orders, a policy (M4) chooses the action, and a builder (M5) re-checks Gate A, signs and submits the batch. Three facts shape the design: the batcher sees the chain only partly, the pool lock is the one shared resource, and capacity limits come from the protocol, so they are enforced by construction rather than learned.

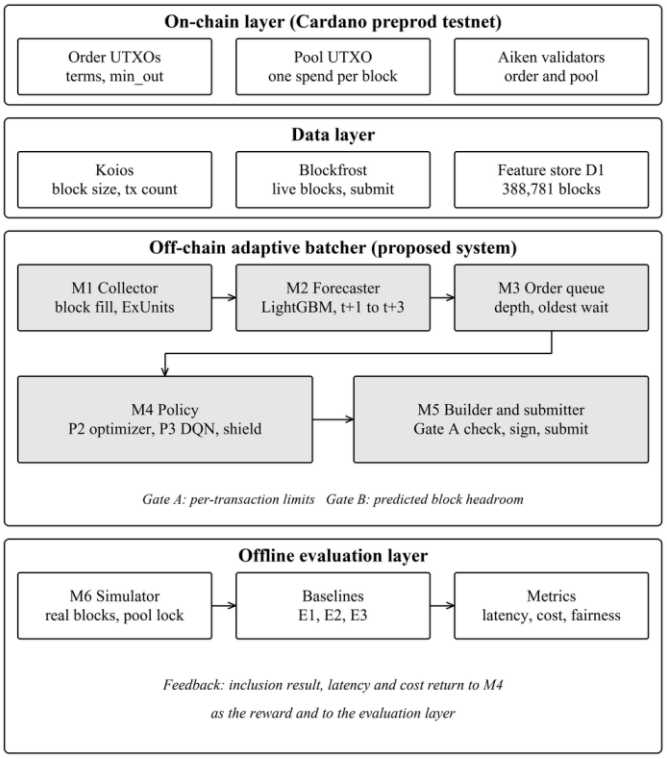


Fig. 1. Architecture of the proposed system: the on-chain layer, the data layer, the off-chain adaptive batcher and the offline evaluation layer.

Fig. 1 shows the four layers of the system. Orders and the pool live on chain, the data layer collects block history from Koios and Blockfrost, and the off-chain batcher (M1 to M5) turns that history into a forecast and a batching decision. The offline layer replays the same blocks through the simulator, so every policy is tested on identical data.

C. Congestion Forecaster

For each block t we build a feature vector x_t from the fill, execution units and transaction counts of the last 20 blocks, their rolling statistics and the time of day. A LightGBM model [7], which builds an ensemble of decision trees one at a time, each new tree correcting the errors left by the earlier ones, and which is trained on a chronological split so that the future never leaks into training, predicts the fill of the next three blocks:

$$\hat{f}_{t+h} = g_{\text{LGBM}}(x_t), \quad h = 1, 2, 3 \tag{3}$$

In (3), g_{LGBM} is the trained model; it maps the feature vector x_t to $\hat{f}_{t+h}$, the predicted fill of block $t+h$ as a fraction of the block limit. A moving average over the last 20 blocks is the reference the forecaster must beat. The forecast is a secondary, defensive input: it lets the policy avoid the rare full block and wait for a quieter one.

D. Decision Rule (P2)

The policy P2 is a forecast-driven rule whose decision flow is shown in Fig. 2. Gate A caps the batch by the per-transaction limits and Gate B by the room predicted in the next block:

$$n_A = \max\{n : s(n) \le 16{,}384,\ m(n) \le 14\times10^6\} \quad (4)$$

$$n_B = \max\{n : \hat{f}_{t+1}\cdot S_{\max} + s(n) \le S_{\max}\} \quad (5)$$

where $S_{\max} = 90{,}112$ bytes is the block size limit; Gate B applies the same check to memory and steps. Thus (4) gives the largest batch that fits the per-transaction limits on size and memory, and (5) the largest batch that still fits in the room left after the forecast fill of the next block. Let $n^* = \min(n_A, n_B)$ and let w be the wait of the oldest order. If the pool is locked or the queue is empty, P2 waits. If $w \ge D_{\max} = 120$ s, it submits n_A at once, whatever the cost. Otherwise it decides

$$a_t = \text{WAIT} \ \text{ if } n^* < N_{\min} \text{ and } \min(\hat{f}_{t+2}, \hat{f}_{t+3}) < \hat{f}_{t+1} \quad (6)$$
$$a_t = \text{SUBMIT}(n^*) \ \text{ otherwise}$$

so a small batch waits for a quieter block and shares the fixed fee more widely. In words, (6) holds a batch back only when it has fewer than $N_{\min}$ orders and one of the next two blocks is forecast to be emptier than the next one; in every other case it sends n^* orders. $N_{\min}$ sets the trade between cost and delay: $N_{\min} = 1$ turns P2 into greedy submission, and larger values buy cheaper batches with longer waits. The amortization curve (Fig. 4) suggests starting near 8; on the validation split, $N_{\min} = 4$ is the setting that beats the tuned interval batcher on both cost and tail latency, so it is the one reported in Section V. Both gates are enforced inside the rule, so an infeasible batch is never proposed. The rule is deterministic, runs in $O(|Q|)$ and is explainable.

E. Learning Agent and Safety Shield

The agent P3 is a masked DQN [8] that picks between WAIT and SUBMIT(n) for n in {1, 4, 8, 12, 16, 20, 25, 30}; illegal actions are removed before selection. A DQN learns a neural network that scores each action by its expected long-run reward and takes the best legal one. Its state holds the queue depth, the oldest and mean wait, the forecast fill and room, and the recent success rate. Its reward is

$$r_t = -(w_C\hat{C}_t + w_L\hat{L}_t + w_S\hat{S}_t + w_K\hat{K}_t) \quad (7)$$

where the hatted terms are the normalized fee share, squared waiting time, slippage and pool-lock time, so the tail is penalized harder than the mean. The weights w_C, w_L, w_S and w_K set how much each cost counts, and the minus sign turns the total cost into a reward that the agent maximizes. A learned policy can fail outside its training range, so a runtime shield wraps it: a forced submission always uses n_A, states outside the trained range of queue depth or oldest wait are passed to P2, and every action is clamped to legal limits.

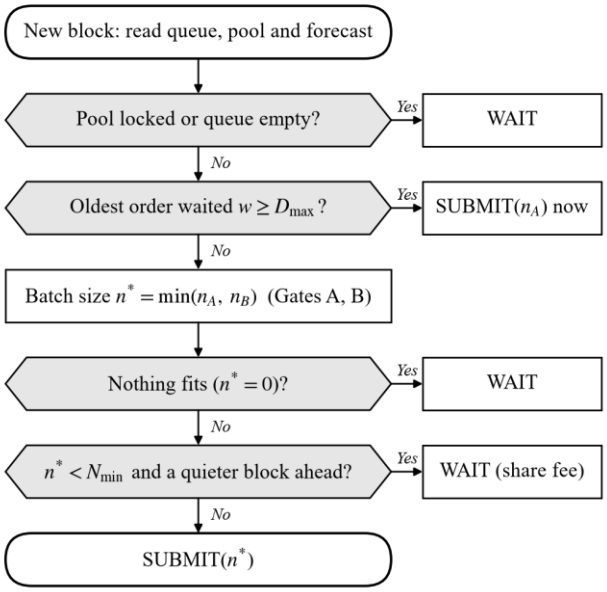


Fig. 2. How the decision rule P2 chooses an action in each block slot.

The rule checks four conditions in order. It waits while the pool is locked or the queue is empty, sends the largest legal batch once the oldest order reaches $D_{\max}$, waits if nothing fits, and holds a small batch only when a quieter block is forecast; otherwise it submits n^*.

F. Chaining and Uniform Clearing

In the simulator a new batch may spend the pool output of a batch still in the mempool, so several batches can leave at one decision point; they resolve in order, and if a parent is rolled back or expires, its children go with it. In the batch builder, each direction clears at one price and opposite directions net against each other, so only the imbalance touches the pool. With a lovelace and b tokens sold into a pool holding x and y, and g = 1 − fee rate, the clearing price is

$$P = g(y + b) / (x + g\cdot a) \quad (8)$$

Equation (8) sets one price from the reserves x, y and the batch totals a, b after the fee factor g, and every order in the same direction receives it. The outcome does not depend on queue order, so no order gains from its place in the queue and the batcher cannot reorder trades for its own profit.

G. Making the Live Batcher Robust

The live batcher reuses the simulator's decision rules and adds safeguards found by reviewing the code against bad inputs and failures. It rejects orders with zero or negative amounts or unreadable addresses. Orders that cannot be filled are set aside, and if a batch fails to build, it is split in half repeatedly to find the bad order without submitting anything. A watchdog frees the pool if a batch's inputs disappear. Chain requests retry with growing, randomized delays, pause after repeated failures and switch to a backup provider, and a batch counts as settled only three blocks deep. The capacity gate also corrects its own size and execution-unit estimates from real transactions.

IV. EXPERIMENTAL SETUP

A. Data

The block history D1 holds 388,781 mainnet blocks over 92 days, collected from Koios with Blockfrost as backup and split by date 70/15/15 into training, validation and test sets. Each row records the block height, slot, size, fill, transaction count and execution units. Fig. 3 shows how full the blocks were: the median block is 3.0% full, 0.56% pass 80% and 0.39% pass 90%. A 30-order batch fails to fit only above about 89% fill, which happens in about one block in 180 and never lasts more than five minutes. The order stream D2 is simulated, with random (Poisson) arrivals, a daily pattern and bursts, at three rates: light, matched and heavy. The measurements D4 are the size, execution units and fee of real batch transactions on the preprod testnet, and they calibrate Gate A.

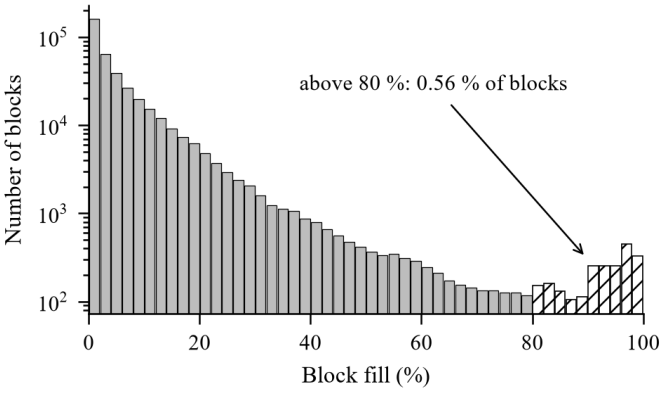


Fig. 3. Distribution of block fill over the 92-day collection window.

The vertical axis is logarithmic, so the tall bars on the left mean that most blocks are nearly empty. The hatched bars above 80% hold only 0.56% of all blocks, which shows that block space rarely limits a batch.

The per-transaction limits of 16,384 bytes and 14 million memory units cap a batch at 24 orders. Fig. 4 shows why batching pays: C_{user} falls steeply with batch size and flattens near ten orders, which bounds the useful range of $N_{\min}$.

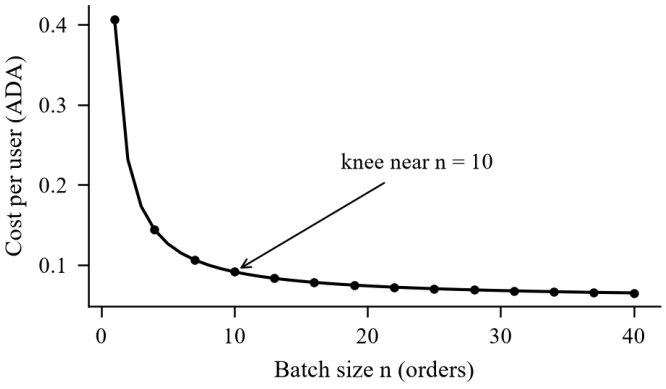


Fig. 4. Cost per user against batch size; the curve flattens near n = 10.

Because the fee has a large fixed part, spreading it over more orders lowers each user's share quickly at first. Beyond about ten orders the saving is small, so waiting for a much larger batch buys little.

B. Simulator, Baselines and Parameters

The simulator replays the recorded blocks slot by slot, with a first-in first-out queue, the pool lock, Gate A and Gate B checks on every submission, batch expiry after 600 slots, order expiry after 3,600 slots and occasional rollbacks. The baselines are E1 (send when M orders have gathered), E2 (send every T seconds) and E3 (greedy: send everything every block), plus a null policy for reference. E1 and E2 were tuned on the validation split (M = 16, T = 20 s), and we report the tuned values so the comparison is fair. The test split was used once. Table II lists the main parameters. The simulator is slightly pessimistic, since a real block producer might drop a marginal transaction to fit the batch.

TABLE II. PRINCIPAL PARAMETERS.

Parameter	Value	Role
$D_{\max}$	120 s	Deadline for the oldest order
$N_{\min}$	4 (swept 1–20)	Amortization threshold
Forecast horizon	3 blocks	Policy lookahead
Moving-average window	20 blocks	Forecast reference
Batch / order TTL	600 / 3,600 s	Expiry intervals
DQN training	200,000 steps	5 seeds (0 to 4)
Evaluation	100 paired episodes	Per rate, test split

Table II lists the settings shared by every experiment. $D_{\max}$ and $N_{\min}$ set the trade between waiting and cost, the TTLs set when batches and orders expire, and the last two rows fix how the agent was trained and how the policies were evaluated.

C. Metrics and Statistics

Latency runs from the arrival of an order to its settlement and is reported as the mean (L-mean) and the 95th percentile (L-p95, the slowest 5% of orders) in seconds; one slot is one second. Cost per user (C-user) is C_{user} from (2), in lovelace. The expiry rate (X-rate) is the share of orders that expire unsettled, and fairness is the Jain index [9] over per-order waits. The index $(\Sigma w_i)^2 / (N \cdot \Sigma w_i^2)$ equals 1 when every order waits equally and falls toward $1/N$ as waits become uneven. An expired order counts at its wait until expiry, so an unsafe policy cannot hide its failures by dropping orders from the average. Policies are compared on paired episodes with 95% bootstrap confidence intervals, and Wilcoxon tests with Holm correction are applied to the family of comparisons. The bootstrap resamples paired episodes to bound each median, the Wilcoxon signed-rank test checks that paired differences favour one side, and Holm correction guards against making many comparisons.

V. RESULTS AND DISCUSSION

A. Congestion and Forecast Accuracy

The premise check comes first. Capacity rarely binds, and the limit on throughput is structural: one batch per decision point, through the pool lock, times the Gate A maximum of 24 orders. On the test split LightGBM has a lower mean absolute error than the moving average at every horizon (Table III and Fig. 5) and calls 74.2% of changes in the right direction. The

root-mean-square errors are close, so the gain is in the typical error and in direction, not in rare spikes. A perfect forecast adds little to the policy: an oracle forecast gives a mean latency of 59.8 s against 60.9 s for P2 at matched load, as expected when capacity seldom binds.

TABLE III. FORECAST ACCURACY ON THE TEST SPLIT (FILL FRACTION).

Model	MAE t+1	MAE t+3	RMSE t+1	Direction
LightGBM	**0.0458**	**0.0466**	**0.0836**	**74.2%**
Moving average	0.0516	0.0524	0.0836	71.0%
Historical mean	0.0607	0.0607	0.0887	66.2%
Last value	0.0607	0.0639	0.1004	n/a

Lower errors are better, and Direction is the share of changes called correctly. LightGBM is best or tied in every column.

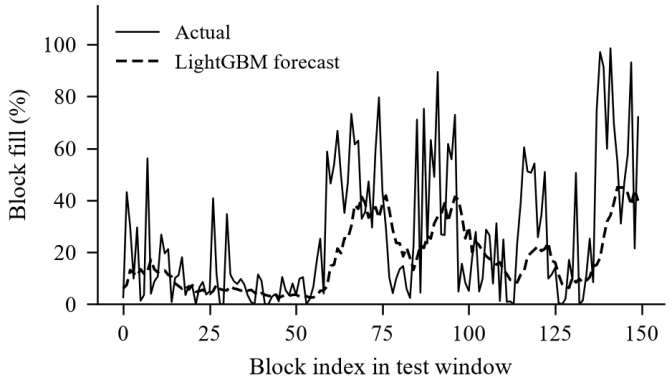


Fig. 5. Forecast against actual block fill over a 150-block window of the test split.

The forecast (dashed) tracks the level and direction of the measured fill (solid) but smooths out sharp spikes.

B. Cost and Latency

Table IV and Table V compare the policies at the matched and heavy arrival rates, as medians over 100 paired episodes. At matched load, P2 cuts C-user by 18.9% against greedy submission (127,738 against 157,445 lovelace) for 5.1 s more average wait (60.9 s against 55.8 s). Against the tuned interval batcher E2 it is faster on average (60.9 s against 63.4 s, with non-overlapping confidence intervals) and 1.5% cheaper. At the light rate the trade is larger, 25% cheaper for 20% more average wait; at the heavy rate it is 12% cheaper for 3% more. No adaptive policy beats greedy on tail latency, because on this chain capacity rarely binds and the gain to be had is in cost and fairness.

TABLE IV. TEST SPLIT, MATCHED ARRIVAL RATE (MEDIANS).

Policy	L-mean	L-p95	C-user	X-rate
Null policy	116.7	191.0	79,451	0.00
E1 size (M = 16)	106.0	183.3	82,906	0.00
E2 interval (T = 20)	63.4	130.5	129,729	0.00
E3 greedy (= P2, N = 1)	55.8	119.0	157,445	0.00
P2 (N = 4)	**60.9**	**125.0**	**127,738**	**0.00**
Oracle (N = 4)	59.8	124.0	124,567	0.00
DQN (unshielded)	88.9	158.6	89,704	0.00

Reading Table IV. Lower is better in all four columns. Greedy (E3) is fastest but dearest, since it sends small batches every block, while the size trigger E1 is cheap but slow because orders wait until 16 have gathered. P2 sits in between, faster and cheaper than the tuned E2, and the oracle (P2 fed the true future fill) is only about one second faster than P2.

TABLE V. TEST SPLIT, HEAVY ARRIVAL RATE (MEDIANS).

Policy	L-mean	L-p95	C-user	X-rate
E1 size (M = 16)	253.5	1,202.8	78,638	0.00
E2 interval (T = 20)	88.7	253.5	95,427	0.00
E3 greedy (= P2, N = 1)	66.2	162.0	116,556	0.00
P2 (N = 4)	**68.0**	**164.5**	**103,022**	**0.00**
DQN (unshielded)	220.9	863.3	85,187	0.07

Table V repeats the comparison at the heavy rate. The size trigger E1 builds a long tail (L-p95 of 1,202.8 s) and the unshielded DQN lets 7% of orders expire, while P2 stays within 2.5 s of greedy on the tail and charges each user 12% less.

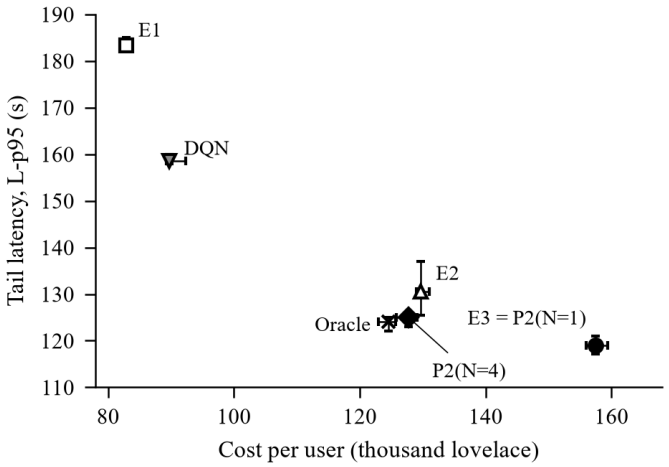


Fig. 6. Tail latency (L-p95, s) against cost per user at the matched rate, with 95% confidence intervals; lower-left is better.

Reading Fig. 6. Each point is one policy, and lower-left is best. P2 sits below and to the left of E2, beating it on both counts; only greedy waits less, and it charges each user 23% more. E1 and the DQN are cheapest but have the longest tails, and the oracle almost overlaps P2 because block space rarely runs out.

C. Behaviour Under Load

The unshielded DQN reaches a low cost but pays for it in waiting (L-p95 158.6 s against 125.0 s for P2 at matched load), and it fails at heavy load, where its L-p95 is 863 s and 7% of orders expire on the test split (Fig. 7). On validation days at heavy load the recorded agent let 35.7% of orders expire at the median and 58.6% in the worst case. The size trigger E1 also collapses there, with an L-p95 of 1,203 s, while P2 stays within 2% of greedy (164.5 s against 162.0 s).

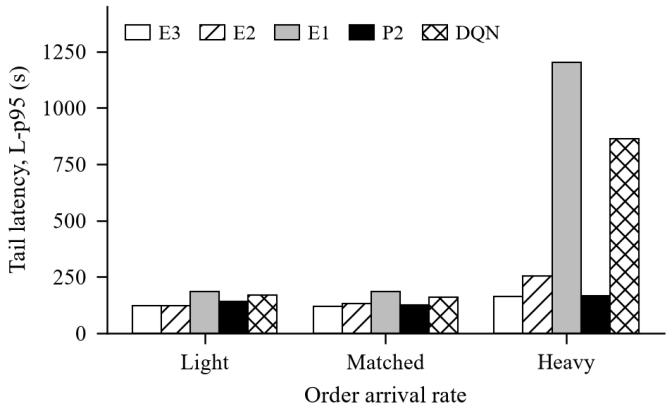


Fig. 7. Tail latency across the light, matched and heavy arrival rates.

All policies stay close at light and matched load, but at heavy load E1 and the unshielded DQN rise sharply while P2 stays level with greedy submission (E3).

TABLE VI. HEAVY-RATE VALIDATION: EFFECT OF THE SAFETY SHIELD.

Policy	**L-p95**	**Expired**	**C-user**
DQN as recorded	3,605	35.7%	116,445
DQN + largest batch	255.0	0%	106,861
Shield (DQN to P2)	**245.5**	**0%**	**79,487**
P2 (N = 4)	245.5	0%	100,662

Reading Table VI. Each row adds one safeguard to the recorded agent. Forcing the largest legal batch at the deadline stops all expiries, and passing unfamiliar states to P2 also gives the lowest cost of any safe option.

D. Chaining and Testnet Demonstration

A diagnosis on validation days found that no batch expired behind a full block at heavy load, while 7–10% of submissions left a backlog larger than one Gate A batch. The ceiling was therefore one batch per decision point, not congestion, which is why a deadline-aware policy alone did not help and chaining did. Allowing two chained batches nearly halves the heavy-load L-p95, from 245.5 s to 120.0 s, and cuts the share of orders settled after $D_{\max}$ from 16.4% to 5.0%, for 1.2% more cost. On the preprod testnet the batcher settled a batch transaction (one order, fee 322,697 lovelace) against the deployed pool validator. The transaction measured 2,682 bytes against an estimate of 800, which is why the capacity gate learns size corrections from real transactions instead of trusting a fixed estimate.

E. Discussion and Limitations

The main finding is a correction. The starting claim of a congested chain was false: blocks are mostly empty, so forecasting capacity helps only as a safeguard, and the real limit is the single lock on the pool. Greedy submission is therefore already close to the best for waiting time. The useful job of an adaptive policy is to trade a few seconds for fewer, fuller batches, and so a lower cost per user, and to stay safe when load rises. The results have limits. They come from simulation on one 92-day window and one pool, with a generated order stream. Chaining exists only in the simulator; the live batcher still locks the pool until settlement. The capacity gate's error matching has not been tested against real Ogmios or Blockfrost error messages, and the testnet run settled a one-order batch. Both validators trust a single batcher key, and orders carry no on-chain deadline. Finally, one-price clearing spreads price impact across the whole batch by design; this is what makes a batch auction fair, but it is still a choice.

VI. CONCLUSION AND FUTURE WORK

This paper treated batching on a Cardano exchange as control of a shared pool lock under uncertain confirmation time. A measurement of 388,781 mainnet blocks showed that the bottleneck is the pool lock, not congestion. Against tuned static batchers, a forecast-driven rule cuts the cost per user by about 19% at matched load for about 5 s more average wait, a learned agent is safe only behind a shield, and no adaptive policy beats greedy submission on tail latency on today's chain. The contributions are a measured correction of the premise, a simulator and evaluation protocol with disclosed baseline tuning, a hard-gated policy family with a safety shield, chaining and one-price clearing, and a working preprod demonstration.

Future work follows from these limits: run chaining in the live batcher and calibrate Gate A on live multi-order batches; test on more pools, longer windows and real order flow; and repeat the study as protocol parameters change, since bigger blocks or faster confirmation would change where capacity binds.

REFERENCES

[1] M. Chakravarty, J. Chapman, K. MacKenzie, O. Melkonian, M. Peyton Jones, and P. Wadler, "The extended UTXO model," in *Financial Cryptography and Data Security Workshops (FC)*, 2020.

[2] H. Nguyen Quang, "Concurrent & deterministic batching on the UTXO ledger," MELD, Tech. Rep., 2021.

[3] P. Bruhwiler, "A concurrent DEX on Cardano," B.Sc. thesis, Univ. of Bern, Switzerland, 2021.

[4] T. Roughgarden, "Transaction fee mechanism design: An economic analysis of EIP-1559," in *Proc. ACM Conf. Economics and Computation (EC)*, 2021.

[5] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in *Proc. ACM Workshop on Hot Topics in Networks (HotNets)*, 2016.

[6] G. Angeris and T. Chitra, "Improved price oracles: Constant function market makers," in *Proc. ACM Conf. Advances in Financial Technologies (AFT)*, 2020.

[7] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[8] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[9] R. Jain, D. Chiu, and W. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," Digital Equipment Corp., Tech. Rep. DEC-TR-301, 1984.